组件

创建 vue文件 在components中创建 注意名字驼峰命名

```
1
2
3
4 <template>
5   <div id="app">
6       <!--3 // 在页面中渲染     像使用html一样使用组件-->
7       <ACtion></ACtion>      //直接使用组件名即可
8
9
10  </div>
11 </template>
12
13 <script>
14
15 //1.es6引入组件方法
16 import  ACtion from "./components/ActineStr.vue"
17
18
19  // 2.在 components中引入
20  components:{
21      ACtion,
22  }
23 </script>
```

父组件向儿子组件通信

prors在子组件上定义用来存放父组件传递过来的数据

```
export default {
    props:['fcar','fmonth']   //字符就是传过来的变量 名字一定要一样

};
```

父组件通过 v-bind指令给子组件传值

```
  <ComSon :fcar=fcar :fmonth=fmonth ></ComSon>
  //利用v-bind属性 把data中得数据传给子组件
  //命名应该始终小写
   props:{
        fcar:{
            required:true , //必须传值
            default:100,   //默认值
            type:Number //类型
        }
    }   //这样写可以校验值
  props: ['',''] //这样子字符串名就是变量名
       //引入子组件
     import ComSon from "./ComSon.vue";
export default {
    data(){
        return{
            fcar:'劳斯莱斯',
            fmonth:3000

        }
    },
 components: {
    ComSon,     //注册组件
}}
```

子传父是用事件传递

父组件利用v-on监听儿子发射过的事件 通过回调函数拿到值

```
<button @click="add">传过去</button>

methods:{
        add(){
           this.$emit('getvalue',this.name)
           console.log('传过去了');
        }
    }

//////////////////父组件//////////////////
 <ComSon :fcar=fcar :fmonth=fmonth  @getvalue="getva"></ComSon> //@监听自定义的事件

   data(){
        return{
            fcar:'劳斯莱斯',
            fmonth:3000,
            list:''   //定义传过来的值的容器
        }
    },
    methods:{
        getva(value){     //value为子组件传过来的值
                this.list=value  //通过函数取得值随后设置给容器

        }
    },
```

跨组件传值

事件总线

简单来说就是一个空的对象实例 借助于空对象实例 实现兄弟组件中的通信

新建一个utlis文件夹 建立一空的实例

```
//index.js中的内容

import Vue from 'vue'
export default new Vue()



//两个组件都引入创建的对象 （index.js)
//发射数据的组件
<script>
import index from '../utils/index'
export default {
  data () {
    return {
      score: 0
    }
  },
  methods: {
    on () {
      index.$emit('onshow', this.score)   //一个兄弟通过$emit 发射一个请求并带参数
    }
  }

}
</script>
```

接受数据的组件

通过$on来接受数据$on(监听的事件，回调函数)

```
<script>
import index from '../utils/index'   //也引入创建的对象
export default {
```

```
  data () {
    return {
      score: ''
    }
  },

  mounted () {        //生命周期函数（钩子函数）挂在到页面的时候自动执行
    index.$on('onshow', (value) => {
      console.log(value)
      this.score = value
    })
  }
}
```

组件的生命周期

```
//组件的生命周期
 beforeCreate () {
    // 实例创建之前
    console.log(this.score)
  },
  created () {
    // 创建实例 一般用于请求数据
  },
  beforeMount () {
    // 挂载组件之前

  },
  mounted () {
    // 挂载组件 //渲染数据操作dom
  },
  beforeUpdate () {
    // 组件更新之前
  },
  updated () {
    // 组件更新
  },
```

```
  beforeDestroy () {
    // 组件销毁之前  一般用于清除定时器
  },
  destroyed () {
        //销毁组件
  }
      ////////再缓存中使用
activated() {
              //组件激活
  },
  deactivated() {

  },             组件失活


```

如果遇到父子组件的顺序

父组件befo create create 子组件 befor -create create befo mounted 父组件 mounted

vue中使用axios

```
1. npm install axios --save // 下载axios
2. 创建api文件夹 和reqiest.js文件

3.  import axios from 'axios' //引入axios
4.创建全局地址  index.js
import axios from 'axios'
export default axios.create({
  baseURL: 'http://localhost:3000'

})

5.在使用的时候引入 创建getbook。js
import axios from '../utils/requirest.js'

export const getbook = () => axios.get('/books')
```

```
export const addboks = (obj) => axios.get('/books', obj) //吐过有对象
6.//使用
import { getbook } from '../api/getbooks'

 async created () {
    const res = await getbook()
    console.log(res)
  }

```

默认插槽

[类似电脑提前给定一个接口，可以接入u盘等设备；使用插槽就可以直接把值插入提前准备好的的槽中]

```
 <slot></slot> //提前定义好一个接口 再子组件内部定义
 <templale>直接写入数据</template>  //调用
```

具名插槽

就是给插槽一个具体的名字让插入的数据再指定位置插入，这样使其插槽更加灵活根据位置进行改变

```
<template slot="header">     //vue 2.6之前可以使用v-slot:name改变但是已经废弃现在通过 外部添加temolate模板进行添加
//slot=name 定义好的名字
      <div class="header">卡片名称
        <a href="#">操作按钮</a>
      </div>
    </template>
//子组件
 <div class="divs">
      <slot name="header"></slot>    //提前定义好插槽的位置，给定name 这样可以指定位置
      <slot></slot> //如果不指定会插入到默认插槽的位置

  </div>
```

通过*$slots* 获取属性及其dom节点，通过name属性定义名字

作用域插槽

父组件为了接受子组件上绑定的数据 必须在v-slot ： 名字后面加上等号赋值一个变量，可以拿到子组件slot上面所有的属性
将组件数据通过v-bind指令绑定到slot组件上这样做的目的为了把组件内部的数据传递到父组件的插槽中去，父组件怎么接收?

```
因为插槽不能直接赋值，说以需要从父组件中把值插过来
通过默认插槽可以定义属性，把值再传给父组件，随后父组件把值再插回子组件，这样可以动态的修改组件
```

ref

在需要操作的的dom 身上添加ref属性

```
<template>
  <div>
    <div ref="dd">{{ dask }}</div>

    <p>davb jhavhjdvh</p>
    //在需要操作的的dom 身上添加ref属性
    <button @click="getDom" ref="dom">获取dom,并且打印数据</button>
  </div>
</template>

<script>
export default {

  data () {
    return {
      dask: 'hello'
    }
  },
  methods: {
```

```
    getDom () {
      this.dask = 'word '
      console.log(this.$refs)
      this.$nextTick(() => {
        console.log(this.$refs.dd)
      })
    }
  }
}

</script>
```

动态组件

内置的有component内置的

```
<component is="comp"></component>//是vue自带的内置组件（展示组件）
//内置组件都有个is属性，is属性是谁，那谁就渲染到这里
//注意在data中定义数据
```

缓存组件

```
<keep-alive></keep-alive>//缓存组件
//要缓存的组件放入缓存组件即可，这是vue的一个内置组件，不需要重复的使用钩子函数，只需要运行于一次，以提升性能
```

自定义指令

一些自己自己定义的指令只要添加上就自动执行

```
<div v-dask="ishow">hduiabjigcaj</div>//v-dask为自己自定义的指令
//自定义指令可以操作dom 以v-自己命名 自定义名字时候不要带v-   但是使用时要带上
//
directives: {
    //el 为 当前的的元素
    //binding 当前元素的信息相当于 sonsile.dir()
    dask: function (el, binding) {

        if (!e.tag === 'div') {
          binding.value = ''     //可以操作dom元素完成想要的操作
          console.log(11)
        }
        }
      })
    }
  }
  unbind: {
      //解绑事件
      }
```

Vue动画

为组件添加动画效果

要过度的元素用 transition标签包裹
需要一个名字name 用这个名字定义css类名的前缀

```
<transition name="ison" ><p v-if="isshow" class="animate__animated animate__shakeY"
>guodu </p></transition>
```

```
style scoped>
.ison-enter {
  /* 过度开始时候的状态 */
```

```
    transition: opacity .5s;
}
.ison-enter-active{
  transition: all 3s;
  /* 过度生效的状态 */
}

.ison-enter-to{
opacity: 0;
/* 过低结束时候的状态 */
}
.ison-leave{
  /* 过渡离开的开始状态 */
}
.ison-leave-avtive{
  /* 过渡离开生效的状态 */

}
.ison-leave-to{
/* 过度离开结束的状态 */
}
</style>
```

animate 动画库 看官方文档的使用方法即可

https://animate.style/